

OBJECT ORIENTED ANALYSIS AND DESIGN

AI-Enabled Smart Campus Management System

(AISCAMS)

OOAD Assignment Report

Student: Haritha A

Register No.: 192321147

Course Code: CSA11

Programme: B.Tech Information Technology

Institution: SIMATS Engineering

Academic Year 2026–2027

DECLARATION

I, Haritha A (192321147), certify that this assignment has been prepared as an original academic submission for Object Oriented Analysis and Design. The analysis, modelling decisions, explanations, and system-specific descriptions in this report have been written for the AISCams case study. External references, if added during implementation, should be acknowledged appropriately.

Signature: _____

Date: _____

1. Problem Analysis and Requirement Summary

AISCaMS is proposed as a centralized campus platform that connects academic, administrative, infrastructure, and student-support services. The system gives different campus roles access to functions appropriate to their responsibilities. Instead of maintaining isolated processes for bookings, events, service requests, attendance, infrastructure monitoring, and reports, the platform provides a common object-oriented model and shared data services.

The design follows the assignment scenario: students can manage schedules, classroom/facility bookings, events and service requests; faculty can manage classes, attendance and classroom requirements; administrators can manage campus resources and reports; security personnel can monitor campus facilities; maintenance staff can process repair requests; and IoT devices can provide occupancy, environmental and equipment-status information.

1.1 System Goals

- Provide role-based access to campus services through a unified platform.
- Reduce conflicts in classroom and facility reservations.
- Support academic activities such as schedules, classes and attendance.
- Allow students and staff to submit and track service requests.
- Monitor selected campus infrastructure through IoT-enabled devices.
- Use intelligent recommendations to rank suitable classrooms, facilities and resources.
- Maintain traceable records for bookings, requests, events and infrastructure status.
- Generate management reports that support operational decisions.

1.2 Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	The system shall authenticate users and provide access according to their role.
FR-02	Role Management	The system shall distinguish Student, Faculty, Administrator, Security Officer and Maintenance Staff permissions.
FR-03	Academic Schedule	Students and faculty shall view relevant academic schedules and class information.
FR-04	Class Management	Faculty shall create/update class details and record attendance.
FR-05	Resource Management	Administrators shall create, update and deactivate campus resources.
FR-06	Resource Booking	Authorized users shall search availability and reserve classrooms/facilities.
FR-07	Booking Control	The system shall prevent overlapping reservations for the same resource and time.
FR-08	AI Recommendation	The system shall rank suitable resources using availability and usage-related criteria.
FR-09	Event Management	Users shall view campus events and register/cancel registration where applicable.
FR-10	Service Requests	Students/faculty shall submit service requests and track their status.
FR-11	Maintenance Processing	Maintenance staff shall accept, update and close assigned requests.
FR-12	Infrastructure Monitoring	The system shall receive and record selected IoT occupancy/environment/equipment readings.
FR-13	Security Monitoring	Security personnel shall view relevant campus facility and status information.
FR-14	Notifications	The system shall provide booking, event and service-request status notifications.
FR-15	Reporting	Administrators shall generate operational reports from stored campus data.

1.3 Non-Functional Requirements

ID	Quality Attribute	Requirement
NFR-01	Security	Authentication, authorization and role-based access shall protect campus functions.

ID	Quality Attribute	Requirement
NFR-02	Performance	Common operations such as schedule lookup and availability search should respond promptly under normal campus load.
NFR-03	Reliability	Bookings and service-request updates should be stored consistently without duplicate or conflicting states.
NFR-04	Scalability	The architecture should permit additional services, resources, users and recommendation features.
NFR-05	Maintainability	Services and domain classes should have clear responsibilities and low unnecessary coupling.
NFR-06	Usability	Interfaces should present role-relevant actions clearly and minimize unnecessary navigation.
NFR-07	Availability	Core campus services should remain usable during normal operating hours with suitable recovery mechanisms.
NFR-08	Traceability	Important actions should be associated with user, timestamp, resource/request and resulting status.

1.4 System Boundary

Inside the AISCaMS boundary are authentication, academic services, resource/booking services, event services, service-request processing, infrastructure monitoring, recommendations, notifications and reporting. External entities include users, IoT devices and external notification infrastructure. The boundary is intentionally limited so that AISCaMS remains responsible for campus business rules rather than device firmware or external communication infrastructure.

2. Actor Identification

Actor	Role in AISCAMS	Major Interactions
Student	Consumes campus services and submits requests.	Login, schedule, booking, event registration, service request, recommendations.
Faculty	Manages academic activities and classroom requirements.	Login, class management, attendance, resource booking/request, service request.
Administrator	Controls campus resources and operational information.	Resource management, infrastructure monitoring, reports, access oversight.
Security Officer	Monitors campus facilities and relevant operational status.	Login, campus monitoring, facility status review.
Maintenance Staff	Handles physical maintenance work.	View assigned requests, update progress, resolve requests.
IoT Device	Provides machine-generated infrastructure readings.	Send occupancy, environmental or equipment-status readings.

2.1 Actor Relationships and Permissions

All human actors inherit the common authentication behaviour represented by the User abstraction. Their specialized permissions are enforced through role-aware services rather than by allowing every role to invoke every operation. This supports encapsulation and keeps authorization rules close to the relevant application boundary.

3. Use Case Descriptions

The following use cases are selected from the requirements and are used as the behavioural basis for the UML models.

UC-01 – Authenticate User

Item	Details
Primary Actor(s)	Student, Faculty, Administrator, Security Officer, Maintenance Staff
Purpose	Authenticate credentials and establish a role-aware session.
Main Flow	1. User enters credentials. 2. System validates credentials. 3. System determines role. 4. System creates authenticated session. 5. Appropriate interface is shown.
Alternative / Exception	Invalid credentials produce an error and no protected operation is exposed.
Requirements	FR-01, FR-02

UC-02 – Book Classroom/Facility

Item	Details
Primary Actor(s)	Student, Faculty
Purpose	Reserve an available campus resource for a specified time.
Main Flow	1. User selects resource type/date/time. 2. System checks availability. 3. Recommendation engine may rank alternatives. 4. User selects an option. 5. Booking is created. 6. Confirmation is issued.
Alternative / Exception	If unavailable, the system suggests other eligible resources/times.
Requirements	FR-06, FR-07, FR-08, FR-14

UC-03 – Manage Class and Attendance

Item	Details
Primary Actor(s)	Faculty
Purpose	Maintain class information and record attendance.
Main Flow	1. Faculty selects class. 2. System loads roster. 3. Faculty records attendance. 4. System validates and saves the attendance record.
Alternative / Exception	Unauthorized user cannot modify attendance.
Requirements	FR-03, FR-04

UC-04 – Submit Service Request

Item	Details
Primary Actor(s)	Student, Faculty
Purpose	Report a campus service or maintenance problem.
Main Flow	1. User selects category. 2. Enters description/location. 3. System assigns priority rules. 4. Request is stored. 5. User receives request identifier.
Alternative / Exception	Missing required information prevents submission.
Requirements	FR-10, FR-14

UC-05 – Process Maintenance Request

Item	Details
Primary Actor(s)	Maintenance Staff
Purpose	Track an assigned request from pending through resolution.

Item	Details
Main Flow	1. Staff opens assigned request. 2. Accepts it. 3. Marks work in progress. 4. Records resolution. 5. Request is closed after verification.
Alternative / Exception	A resolved request may be reopened if the issue is reported again.
Requirements	FR-11

UC-06 – Monitor Infrastructure

Item	Details
Primary Actor(s)	Administrator, Security Officer
Purpose	View current and recorded infrastructure information.
Main Flow	1. User selects facility/device category. 2. System retrieves recent readings/status. 3. Status is displayed. 4. Abnormal information can be reviewed.
Alternative / Exception	Unavailable device data is marked as stale rather than treated as a current reading.
Requirements	FR-12, FR-13

UC-07 – Manage Campus Resource

Item	Details
Primary Actor(s)	Administrator
Purpose	Create, update, deactivate and inspect campus resources.
Main Flow	1. Administrator selects resource. 2. Edits details or status. 3. System validates the change. 4. Change is stored and reflected in future availability searches.
Alternative / Exception	A resource with active reservations should not be silently deleted.
Requirements	FR-05

UC-08 – Generate Operational Report

Item	Details
Primary Actor(s)	Administrator
Purpose	Produce reports about bookings, requests, resources or usage.
Main Flow	1. Administrator chooses report type and period. 2. Reporting service retrieves data. 3. Aggregation is performed. 4. Report is displayed/exported.
Alternative / Exception	Invalid date ranges are rejected.
Requirements	FR-15

4. Use Case Diagram

The diagram defines the AISCAMS system boundary and connects each actor to the functions that actor can initiate. The AI recommendation capability is treated as a system function supporting resource selection rather than as an independent human actor.

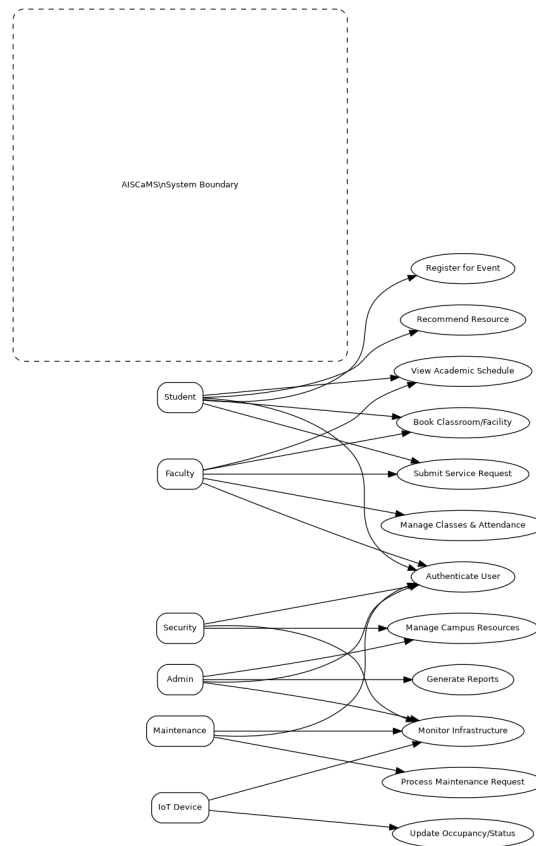


Figure 4.1 AISCAMS Use Case Model

5. Object and Class Identification

5.1 Candidate Objects

Object Category	Examples	Reason for Inclusion
People	Student, Faculty, Administrator, Security Officer, Maintenance Staff	Represent role-specific behaviour and permissions.
Academic	Class, AttendanceRecord, Schedule	Represent teaching and learning activities.
Resources	Classroom, Laboratory, ParkingArea, Equipment	Represent reservable or monitored campus assets.
Transactions	Booking, EventRegistration, ServiceRequest	Represent time-bound campus interactions.
Infrastructure	IoTDevice, SensorReading	Represent device-originated campus information.
Intelligence	RecommendationEngine	Encapsulates resource-ranking logic.
Reporting	ReportGenerator, Report	Encapsulates aggregation and presentation of operational data.

5.2 Main Classes and Responsibilities

Class	Key Attributes	Core Methods
User	userId, name, email, role	login(), logout(), hasPermission()
Student	program, year	viewSchedule(), bookResource(), submitRequest()
Faculty	department	manageClass(), markAttendance(), requestResource()
Administrator	adminLevel	manageResource(), monitorInfrastructure(), generateReport()
CampusResource	resourceId, name, location, capacity, status	checkAvailability(), reserve(), release()
Booking	bookingId, startTime, endTime, status	create(), cancel(), confirm()
ServiceRequest	requestId, category, priority, status	create(), assign(), updateStatus(), close()
RecommendationEngine	modelVersion	recommendResource(), rankOptions()
IoTDevice	deviceId, type, reading	sendReading(), updateStatus()
ReportGenerator	reportType, generatedAt	generate(), export()

6. Class Diagram

The class model applies inheritance to role-specific users and associations to connect bookings, resources, requests, devices, recommendations and reports. The diagram intentionally keeps business operations in domain classes/services rather than placing all behaviour inside a single controller class.

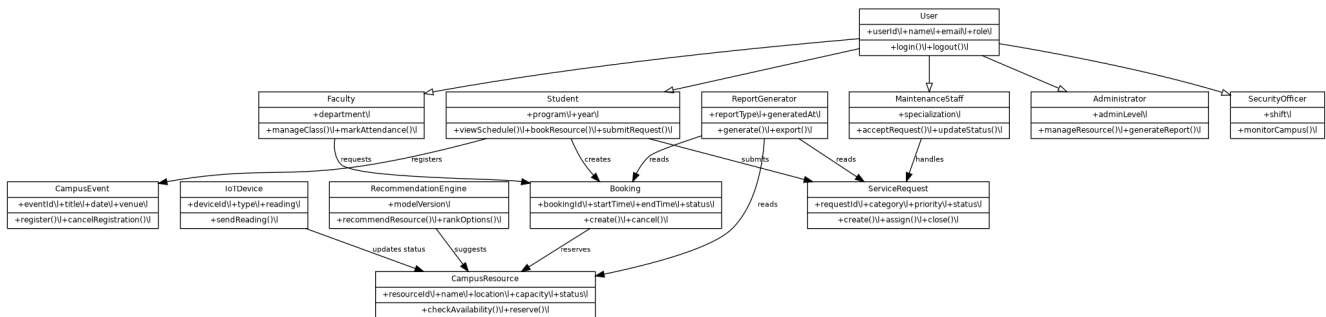


Figure 6.1 Core AISCaMS Class Model

6.1 Relationship Justification

- Generalization: Student, Faculty, Administrator, Security Officer and Maintenance Staff specialize User.
- Association: Booking connects an authorized user to a CampusResource for a time interval.
- Association: ServiceRequest is created by a user and handled by MaintenanceStaff.
- Dependency: RecommendationEngine reads resource information to produce ranked suggestions.
- Dependency: ReportGenerator reads booking, service-request and resource information to build reports.
- IoTDevice updates resource status/readings without becoming responsible for booking or reporting rules.

7. Sequence / Interaction Diagram

The following interaction models a student requesting a classroom recommendation and then making a booking. It demonstrates how the UI, authentication, recommendation, resource and booking responsibilities cooperate.

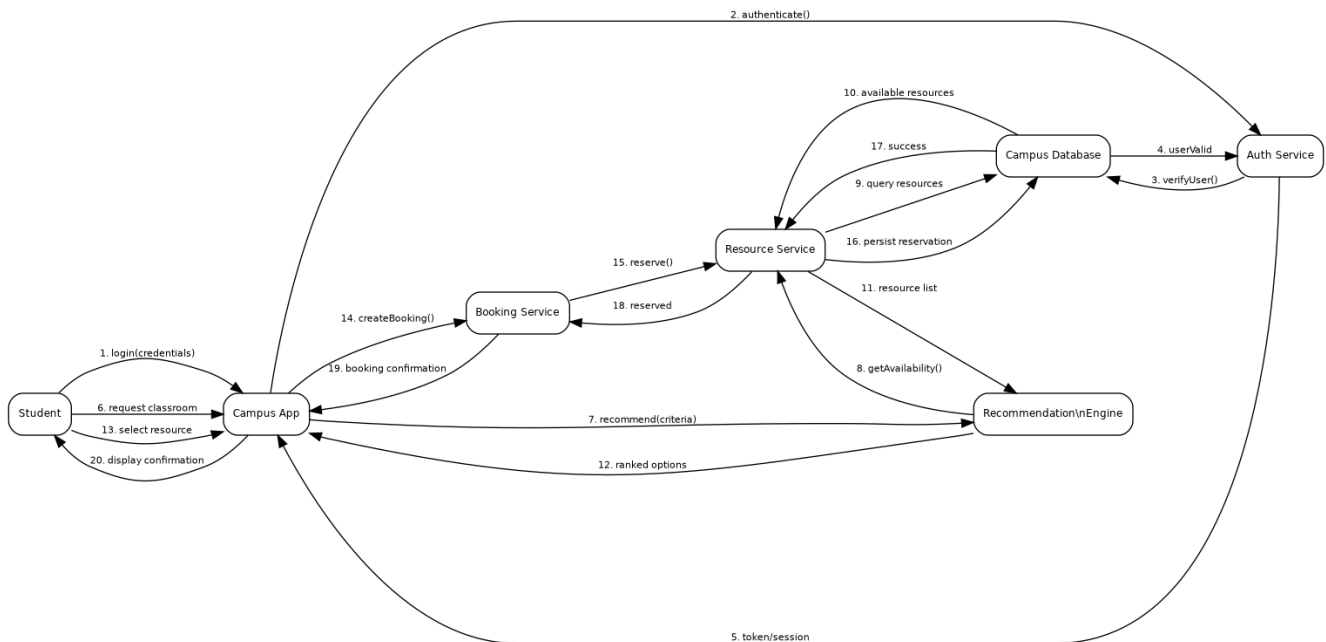


Figure 7.1 Classroom Recommendation and Booking Sequence

The sequence is consistent with the class responsibilities: the recommendation engine ranks options, the resource service verifies availability, and the booking service creates the reservation. No single class is assigned all responsibilities.

8. Activity Diagram

The activity flow covers authentication, resource selection, availability checking, recommendation, reservation and exception handling. A failed authentication loops back to login, while an unsuccessful reservation returns the user to availability checking.

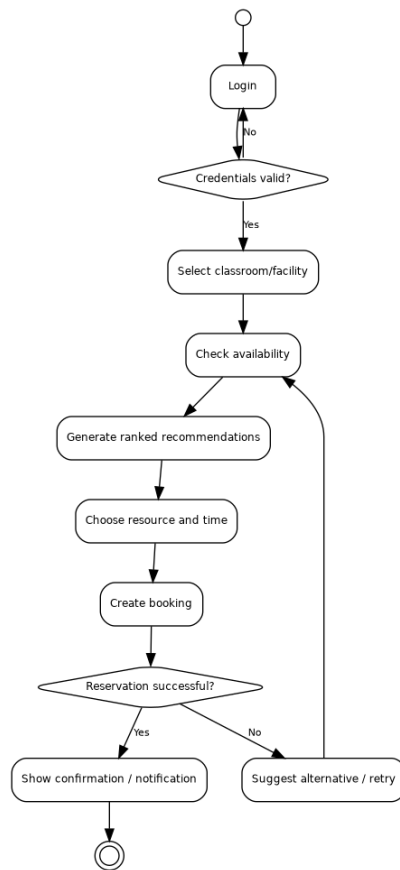


Figure 8.1 Resource Booking Activity Flow

9. State Transition Diagram

A ServiceRequest is a useful stateful object because its lifecycle changes as maintenance work progresses. The model prevents ambiguous status updates and supports traceability from submission to closure.

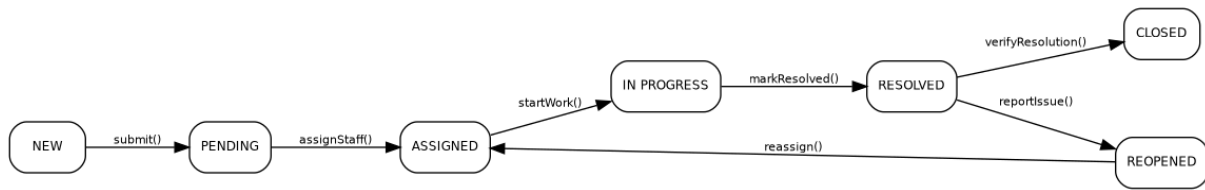


Figure 9.1 Service Request State Model

State rules: NEW requests are submitted but not yet queued for processing; PENDING indicates that the request is awaiting assignment; ASSIGNED identifies responsible maintenance staff; IN PROGRESS indicates active work; RESOLVED records a proposed solution; CLOSED indicates verification; REOPENED represents a returned issue that requires further work.

10. Package Diagram

The package structure separates presentation, authentication, academic services, booking/resource management, event services, maintenance, IoT integration, AI recommendations, reporting and data access. This decomposition supports cohesion inside packages and reduces unnecessary coupling between unrelated functions.

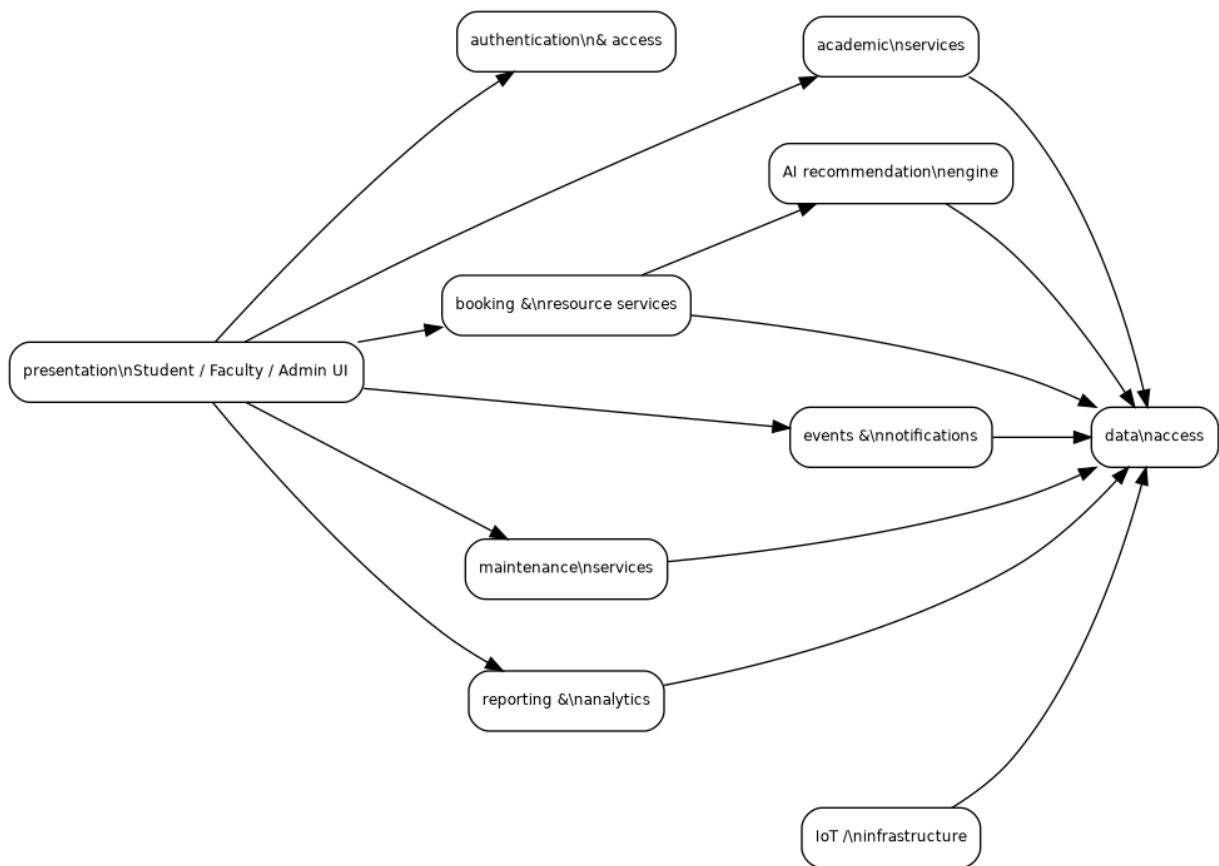


Figure 10.1 AISCAMS Package Structure

11. Component Diagram

The component architecture represents independently deployable or replaceable logical services. An API gateway provides a controlled entry point while domain services own their respective responsibilities. The recommendation engine is separated so that its model can evolve without rewriting booking logic.

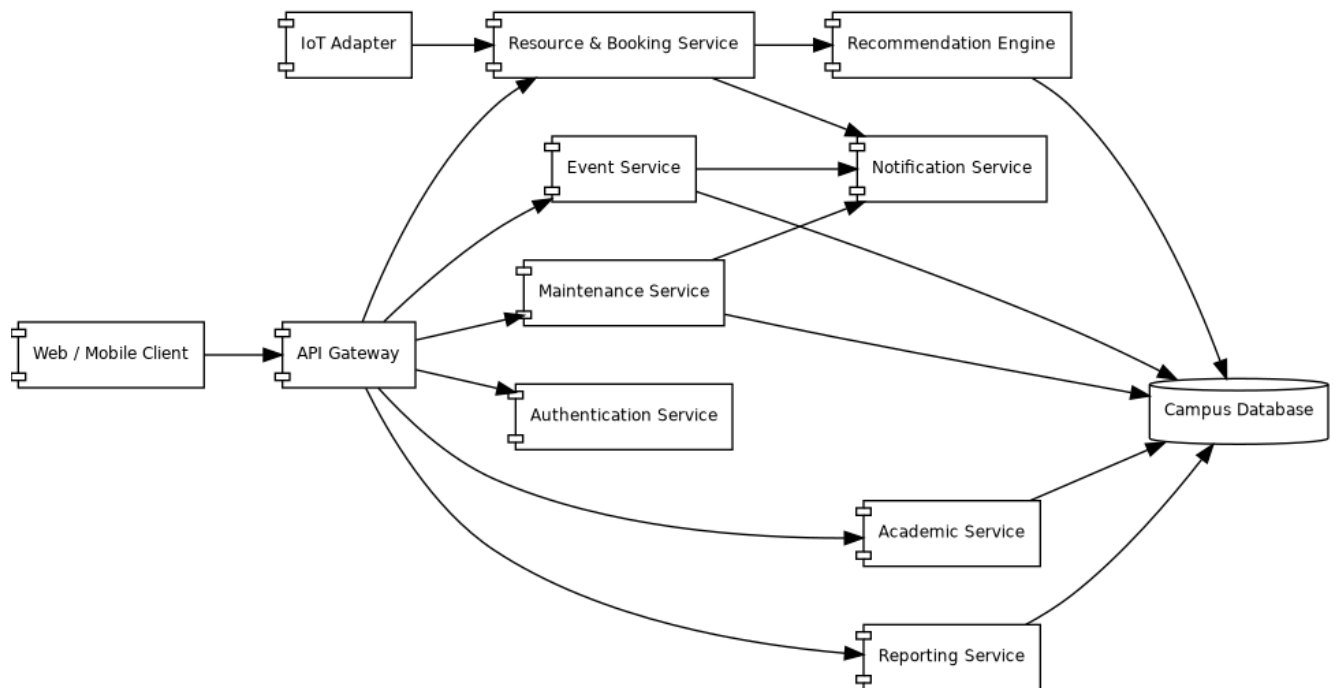


Figure 11.1 AISCaMS Component Architecture

12. Deployment Diagram

The deployment model maps user devices, campus administration workstations, application services, AI inference, database storage, IoT devices and notification infrastructure. Communication protocols are illustrative architectural choices and can be replaced by the technologies selected during implementation.

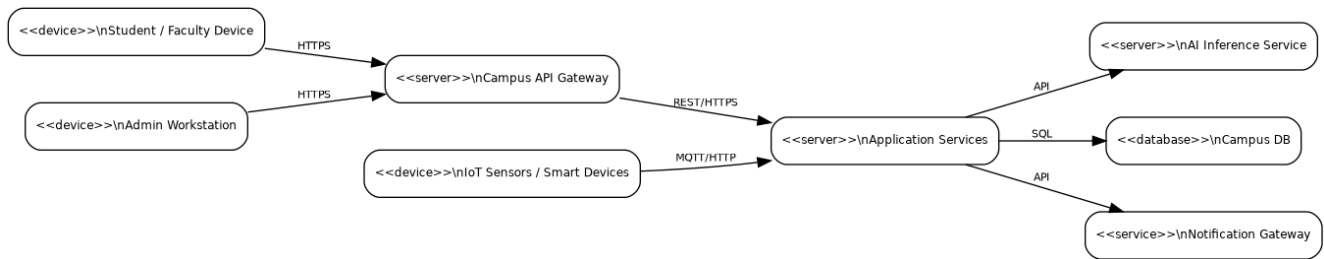


Figure 12.1 AISCaMS Deployment Model

13. Object Responsibility Analysis

Object/Class	Primary Responsibility	Collaborators
User	Maintain common identity and role information.	Authentication Service, role-specific subclasses
CampusResource	Maintain resource details and availability state.	Booking, IoTDevice
Booking	Maintain reservation lifecycle and time interval.	User, CampusResource, Notification Service
ServiceRequest	Maintain maintenance issue lifecycle.	User, MaintenanceStaff, Notification Service
MaintenanceStaff	Perform assigned maintenance workflow.	ServiceRequest
IoTDevice	Publish infrastructure readings/status.	Resource Service / IoT Adapter
RecommendationEngine	Rank eligible resources using criteria and available data.	CampusResource, Booking
ReportGenerator	Aggregate domain data into operational reports.	Booking, ServiceRequest, CampusResource
Administrator	Control campus configuration and operational oversight.	Resource Service, Reporting Service
Notification Service	Deliver event/status messages to users.	Booking, Event, ServiceRequest

13.1 GRASP-Oriented Responsibility Decisions

- Information Expert: CampusResource manages availability-related information because it owns resource state.
- Controller: application services receive user actions and delegate to domain objects rather than implementing every business rule.
- Low Coupling: RecommendationEngine depends on resource availability interfaces instead of directly controlling database details.
- High Cohesion: ReportGenerator focuses on report construction, while Booking remains responsible for booking lifecycle rules.
- Polymorphism: role-specific User subclasses can implement permitted behaviour without duplicating authentication identity data.

14. Design Axioms / Principles and Design Pattern Justification

14.1 Object-Oriented Principles

Principle	Application in AISCaMS
Encapsulation	Booking, ServiceRequest and CampusResource hide their internal state and expose controlled operations.
Abstraction	User and CampusResource represent common concepts while hiding implementation details.
Inheritance	Role-specific users derive common identity and authentication-related properties from User.
Polymorphism	Different user roles can respond to permitted operations according to role-specific rules.
High Cohesion	Each service/class is assigned a focused purpose such as booking, reporting or recommendations.
Low Coupling	Modules communicate through service boundaries and domain interfaces rather than shared internal state.

14.2 Design Patterns

Pattern	Proposed Use	Justification
Strategy	Recommendation strategies can vary by resource type or campus policy.	Allows recommendation logic to change without modifying booking workflow.
Factory Method	Create different CampusResource specializations such as Classroom, Laboratory or ParkingArea.	Centralizes object creation and supports future resource types.
Observer	Notify interested users when booking/request status changes.	Separates state changes from notification delivery.
Facade	Provide a simplified campus-service interface to the client application.	Reduces client knowledge of multiple internal services.
Repository	Encapsulate persistence operations for Booking, Resource and ServiceRequest.	Keeps storage concerns separate from domain rules.

14.3 Axioms / Design Evaluation

- Minimize coupling by preventing domain objects from depending directly on database or UI classes.
- Maximize cohesion by assigning each class a small, understandable responsibility.
- Prefer composition for configurable behaviour and inheritance only where the subtype truly represents an 'is-a' relationship.
- Keep interfaces stable so that AI recommendation or notification implementations can be replaced independently.
- Validate state transitions at the object/service boundary to protect data consistency.

15. Assumptions and Constraints

15.1 Assumptions

- Every human user has a unique campus identity and assigned role.
- Resource administrators maintain reasonably accurate resource details and operating status.
- Booking time zones and academic calendar rules are configured centrally.
- IoT devices, where deployed, provide readings with a device identifier and timestamp.
- The recommendation engine receives enough availability information to rank feasible options.
- Notifications are delivered through a campus-approved communication channel.
- Historical records are retained according to institutional policy.

15.2 Constraints

- The model must remain consistent across all UML diagrams.
- A resource cannot have overlapping confirmed bookings.
- Role permissions restrict access to sensitive academic and administrative operations.
- IoT data may be delayed or unavailable; the system must distinguish stale data from current data.
- The proposed AI recommendation is advisory and should not bypass hard constraints such as availability or authorization.
- The report represents a design/prototype specification; production deployment would require security, performance and integration testing.

16. Requirement-to-Use Case/Class Traceability

Requirement	Primary Use Case(s)	Supporting Class/Component
FR-01 Authentication	UC-01	User, Authentication Service
FR-02 Role Management	UC-01	User and role-specific subclasses
FR-03 Academic Schedule	UC-03	Faculty, Student, Academic Service
FR-04 Class & Attendance	UC-03	Faculty, Class, AttendanceRecord
FR-05 Resource Management	UC-07	Administrator, CampusResource, Resource Service
FR-06 Resource Booking	UC-02	Booking, CampusResource, Booking Service
FR-07 Booking Conflict Control	UC-02	Booking, CampusResource
FR-08 AI Recommendation	UC-02	RecommendationEngine, AI Service
FR-09 Events	Event management flow	CampusEvent, Event Service
FR-10 Service Requests	UC-04	ServiceRequest, Maintenance Service
FR-11 Maintenance Processing	UC-05	MaintenanceStaff, ServiceRequest
FR-12 Infrastructure Monitoring	UC-06	IoTDevice, IoT Adapter
FR-13 Security Monitoring	UC-06	SecurityOfficer, Monitoring Service
FR-14 Notifications	UC-02, UC-04, UC-05	Notification Service
FR-15 Reporting	UC-08	ReportGenerator, Reporting Service

Traceability demonstrates that requirements are not isolated statements: each major requirement maps to at least one user-visible behaviour and one or more model elements. The booking requirement, for example, is reflected in the use case, activity/sequence flow, Booking and CampusResource classes, and the booking/resource components.

17. Implementation / Prototype and Results

17.1 Proposed Prototype Modules

Module	Prototype Responsibility	Representative Output
Login	Authenticate and identify role.	Role-specific dashboard.
Student Dashboard	Schedule, resources, events and requests.	Upcoming classes, available resources and request status.
Faculty Dashboard	Class, attendance and classroom functions.	Class list, attendance controls and resource requests.
Admin Dashboard	Resource configuration, monitoring and reports.	Resource status, usage summaries and report controls.
Maintenance Dashboard	Assigned service requests and status updates.	Request queue and lifecycle status.
Recommendation View	Rank feasible classrooms/facilities.	Recommended resource list with reasons/criteria.
Monitoring View	Display recent infrastructure information.	Occupancy/device status and timestamp.
Reporting View	Aggregate operational information.	Booking, resource usage and service-request reports.

17.2 Illustrative Booking Output

Field	Example
User	Student-1024
Resource	Smart Classroom C-204
Date	2026-09-08
Time	10:00–11:00
Recommendation Basis	Capacity fit + availability + recent usage
Booking Status	CONFIRMED
Notification	Booking confirmation generated

The above values are illustrative prototype data, not measured production results. They demonstrate the information that a working implementation would be expected to display.

17.3 Proposed Test Cases

Test ID	Scenario	Expected Result
T01	Valid student login	Student dashboard opens.
T02	Invalid password	Access denied and error displayed.
T03	Book available classroom	Booking becomes confirmed.
T04	Book already reserved classroom/time	Booking is rejected and alternatives may be shown.
T05	Submit service request	Unique request ID and PENDING status are created.
T06	Maintenance marks request resolved	Request moves to RESOLVED.
T07	IoT status update received	Latest reading/status is stored with timestamp.
T08	Generate monthly booking report	Report contains aggregated booking information for selected period.

18. Reflection on Design Decisions and Learning Outcomes

Developing AISCaMS as an OOAD model demonstrates why a large campus platform should not be represented as one monolithic class. The system contains several independent responsibilities: identity, academic activity, resource booking, maintenance, infrastructure monitoring, recommendations and reporting. Separating these responsibilities makes the model easier to understand and provides clear locations for future changes.

A major design decision was to treat CampusResource and Booking as separate concepts. A resource describes what the campus owns, while a booking represents a temporary reservation of that resource. This separation makes conflict detection and future resource types easier to model. Similarly, ServiceRequest has its own state lifecycle because maintenance work progresses through identifiable stages.

The AI recommendation capability was intentionally modelled as a supporting component rather than allowing artificial intelligence to control core business rules. Availability, authorization and booking validity remain deterministic constraints. The recommendation engine ranks eligible choices, which keeps the system explainable and prevents a recommendation from creating an invalid reservation.

The UML diagrams reinforce one another: actors and use cases establish system behaviour; classes provide the structural vocabulary; sequence and activity diagrams describe behaviour; the state model explains request lifecycle; and package, component and deployment diagrams show modular and physical organization. This consistency is essential for a maintainable OOAD design.

18.1 Learning Outcomes

- Applied object identification, classification and responsibility assignment to a multi-domain problem.
- Used UML to connect requirements, structure, behaviour and architecture.
- Applied encapsulation, abstraction, inheritance, polymorphism, cohesion and coupling considerations.
- Selected design patterns according to concrete design needs rather than adding patterns without purpose.
- Understood the importance of traceability between requirements and model elements.
- Recognized that AI/IoT features should be integrated through clear interfaces and controlled responsibilities.

18.2 SDG Relevance

SDG	Connection to AISCaMS
SDG 4 – Quality Education	Improves access to academic schedules, classrooms, facilities and digital campus services.
SDG 9 – Industry, Innovation and Infrastructure	Uses intelligent recommendations, software services and IoT-enabled monitoring to support modern campus infrastructure.
SDG 11 – Sustainable Cities and Communities	Encourages better utilization of shared spaces, facilities and campus resources through centralized management and monitoring.

19. GitHub Upload and Submission Checklist

The OOAD assignment brief requires a GitHub upload. The repository link should be added after the student creates or updates the repository; no repository URL is invented in this report.

Item	Status for this report	Action before submission
Problem Analysis & Requirements	Completed	Review against final implementation.
Actor Identification	Completed	Verify role permissions.
Use Case Descriptions	Completed	Add any institution-specific use cases if required.
Use Case Diagram	Included	Check diagram readability after upload.
Object/Class Identification	Completed	Align names with source code if implemented.
Class Diagram	Included	Update methods if implementation changes.
Sequence Diagram	Included	Add further sequences if faculty/maintenance flows are implemented.
Activity Diagram	Included	Validate against final booking workflow.
State Diagram	Included	Validate service-request lifecycle.
Package Diagram	Included	Map to final project packages.
Component Diagram	Included	Map to actual services/modules.
Deployment Diagram	Included	Replace illustrative infrastructure with actual deployment if available.
Responsibility Analysis	Completed	Review with implementation classes.
Patterns & Principles	Completed	Keep only patterns actually used in implementation.
Assumptions & Constraints	Completed	Add institution-specific constraints if known.
Traceability	Completed	Update after final requirements changes.
Prototype/Results	Completed as proposed/illustrative	Replace illustrative values with actual test evidence if available.
Reflection & SDG	Completed	Personalize reflection before submission.
GitHub Repository	Pending student URL	Paste repository URL in final submission form.

Recommended repository structure:

```
AISCaMS - OOAD/  
■■■■ README.md  
■■■■ diagrams/  
■■■■ requirements/  
■■■■ src/  
■■■■ test/  
■■■■ report/OOAD_AISCaMS_Report.pdf
```

Conclusion

AISCaMS provides an object-oriented blueprint for integrating academic, administrative, infrastructure and student-support activities within a single intelligent campus platform. The model identifies role-based actors, domain objects, classes and responsibilities, then connects them through consistent UML behavioural and architectural views.

The proposed design emphasizes modularity, maintainability, scalability and traceability. Resource booking, maintenance requests, infrastructure monitoring and AI recommendations are separated into clear responsibilities so that changes in one area do not unnecessarily affect the rest of the system. The resulting model satisfies the required OOAD deliverables while leaving implementation-specific technologies open for the final prototype.

END OF REPORT